

Competitive Programming

Section 14 Editorial: Core Patterns Mixed Contest

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	3
2	A. Complete Window	4
2.1	Restatement	4
2.2	Constraints and traps	4
2.3	Observations	4
2.4	Sample walk-through	4
2.5	Algorithm	5
2.6	Correctness proof	5
2.7	Complexity	5
2.8	Full C++ solution	6
2.9	Full Python solution	7
2.10	Tests that matter	8
3	B. Minimum Split Limit	9
3.1	Restatement	9
3.2	Constraints and traps	9
3.3	Observations	9
3.4	Sample walk-through	9
3.5	Algorithm	10
3.6	Correctness proof	10
3.7	Complexity	11
3.8	Full C++ solution	11
3.9	Full Python solution	12
3.10	Tests that matter	13
4	C. Active Colors	14
4.1	Restatement	14
4.2	Constraints and traps	14
4.3	Observations	14
4.4	Sample walk-through	15
4.5	Algorithm	15
4.6	Correctness proof	15
4.7	Complexity	16
4.8	Full C++ solution	16
4.9	Full Python solution	18
4.10	Tests that matter	19
5	Stress testing	20
6	What to carry forward	21

1 Editorial overview

This mixed contest is about choosing tools.

The intended pattern decisions are:

Problem	Pattern
A. Complete Window	sliding window with frequencies
B. Minimum Split Limit	binary search on answer
C. Active Colors	sweep line with answer restoration

The editorial explains both the algorithm and the recognition step.

2 A. Complete Window

2.1 Restatement

Find the shortest contiguous subarray containing every required label from 1 through m at least once. Values outside this range are irrelevant.

2.2 Constraints and traps

The sum of n is large, so checking every subarray is too slow.

Common traps:

- counting irrelevant labels as useful;
- stopping after finding the first valid window;
- not shrinking the left side after a window becomes valid.

2.3 Observations

The statement asks for a shortest contiguous subarray and the validity condition is based on counts inside the current window. This suggests sliding window.

Maintain:

```
freq[x] = occurrences of required label x inside the window
have = number of required labels whose frequency is positive
```

When `have == m`, the current window is valid. Shrink it from the left to find the shortest valid window ending at the current right endpoint.

2.4 Sample walk-through

Array:

```
5 1 2 9 3 2 1 3
m = 3
```

The window `[1,2,9,3]` is valid with length 4. After shrinking away the left 1, it becomes invalid.

Later, `[3,2,1]` is valid with length 3, which is optimal.

2.5 Algorithm

```
left = 0
have = 0
best = infinity
freq = zeros for labels 1..m

for right in 0..n-1:
    add a[right] if it is required
    while have == m:
        best = min(best, right-left+1)
        remove a[left] if it is required
        left += 1

print best or -1
```

2.6 Correctness proof

Lemma 1. At all times, `freq` and `have` correctly describe the current window `[left, right]`.

Proof. The algorithm increments a required label's count when the right endpoint enters the window and decrements it when the left endpoint leaves. `have` changes exactly when a required label's count moves between zero and one. $\square$

Lemma 2. For each fixed `right`, the algorithm checks every minimal valid window ending at `right`.

Proof. Once the window is valid, the algorithm repeatedly removes the left endpoint and records the length before each removal. It stops exactly when the window becomes invalid. Therefore the last valid window before stopping is the shortest valid window ending at `right`, and all longer valid windows during that shrinking process were also considered. $\square$

Theorem. The algorithm returns the length of the shortest valid subarray.

Proof. Every valid subarray has some right endpoint. By Lemma 2, when that right endpoint is processed, the algorithm considers the shortest valid window ending there, whose length is no greater than the chosen subarray. Taking the minimum over all right endpoints gives the global optimum. $\square$

2.7 Complexity

Each endpoint moves right at most `n` times.

```
Time:  $O(n)$ 
Memory:  $O(m)$ 
```

2.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int solve_case(const vector<int> &a, int m) {
    vector<int> freq(m + 1, 0);
    int have = 0;
    int best = (int)a.size() + 1;
    int left = 0;
    for (int right = 0; right < (int)a.size(); ++right) {
        int value = a[right];
        if (1 <= value && value <= m) {
            if (freq[value] == 0) ++have;
            ++freq[value];
        }
        while (have == m) {
            best = min(best, right - left + 1);
            int old = a[left];
            if (1 <= old && old <= m) {
                --freq[old];
                if (freq[old] == 0) --have;
            }
            ++left;
        }
    }
    return best == (int)a.size() + 1 ? -1 : best;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        vector<int> a(n);
        for (int &x : a) cin >> x;
        cout << solve_case(a, m) << '\n';
    }

    return 0;
}
```

2.9 Full Python solution

```
import sys

def solve_case(a: list[int], m: int) -> int:
    freq = [0] * (m + 1)
    have = 0
    best = len(a) + 1
    left = 0
    for right, value in enumerate(a):
        if 1 <= value <= m:
            if freq[value] == 0:
                have += 1
            freq[value] += 1
        while have == m:
            best = min(best, right - left + 1)
            old = a[left]
            if 1 <= old <= m:
                freq[old] -= 1
                if freq[old] == 0:
                    have -= 1
            left += 1
    return -1 if best == len(a) + 1 else best

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        m = data[idx + 1]
        idx += 2
        a = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(a, m)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

2.10 Tests that matter

```
valid window at the beginning  
valid window at the end  
irrelevant values inside the best window  
missing required label -> -1
```

3 B. Minimum Split Limit

3.1 Restatement

Split an array of positive integers into at most d nonempty contiguous groups. Minimize the largest group sum.

3.2 Constraints and traps

The array order cannot change. Sorting the values is invalid.

Common traps:

- binary searching below $\max(a)$;
- using exactly d groups when the statement says at most d ;
- forgetting that all values are positive, which makes the greedy check valid.

3.3 Observations

Let C be a candidate maximum group sum.

Predicate:

```
feasible(C) = we can split into at most d groups with every group sum <= C
```

If C is feasible, any larger value is also feasible. So we binary search the minimum feasible C .

For a fixed C , greedily fill each group as much as possible before starting a new group. This minimizes the number of groups needed for that C .

3.4 Sample walk-through

Array:

```
7 2 5 10 8  
d = 2
```

Check $C = 18$:

```
group 1: 7+2+5 = 14
next 10 would exceed 18
group 2: 10+8 = 18
groups = 2, feasible
```

Check $C = 17$:

```
group 1: 7+2+5 = 14
group 2: 10
group 3: 8
groups = 3, not feasible
```

Answer: 18.

3.5 Algorithm

```
lo = max(a)
hi = sum(a)
while lo < hi:
    mid = (lo + hi) // 2
    if groups_needed(mid) <= d:
        hi = mid
    else:
        lo = mid + 1
print lo
```

3.6 Correctness proof

Lemma 1. For a fixed capacity C , the greedy scan uses the minimum possible number of groups.

Proof. The greedy scan ends a group only when the next element would exceed C . Any valid solution's first group cannot extend beyond that point, because including that next element would also exceed C . Therefore the greedy first group is at least as long as the first group of some optimal split. Repeating the argument on the remaining suffix proves the greedy scan uses no more groups than any other valid split. $\square$

Lemma 2. The feasibility predicate is monotone.

Proof. If capacity C allows a valid split, then any capacity $C' \geq C$ allows the same split. $\square$

Theorem. The binary search returns the minimum possible maximum group sum.

Proof. The lower bound $\max(a)$ is necessary because every element must fit in some group. The upper bound $\sum(a)$ is feasible by taking one group. By Lemma 2, feasible capacities form a suffix of the search range. The binary search maintains the minimum feasible capacity inside $[lo, hi]$ and terminates when $lo == hi$, which is that minimum. $\square$

3.7 Complexity

Time: $O(n \log \text{sum}(a))$
Memory: $O(1)$ besides input

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int groups_needed(const vector<long long> &a, long long limit) {
    int groups = 1;
    long long current = 0;
    for (long long x : a) {
        if (current + x <= limit) {
            current += x;
        } else {
            ++groups;
            current = x;
        }
    }
    return groups;
}

long long solve_case(const vector<long long> &a, int d) {
    long long lo = *max_element(a.begin(), a.end());
    long long hi = accumulate(a.begin(), a.end(), 0LL);
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (groups_needed(a, mid) <= d) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, d;
        cin >> n >> d;
```

```

        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        cout << solve_case(a, d) << '\n';
    }

    return 0;
}

```

3.9 Full Python solution

```

import sys

def groups_needed(a: list[int], limit: int) -> int:
    groups = 1
    current = 0
    for x in a:
        if current + x <= limit:
            current += x
        else:
            groups += 1
            current = x
    return groups

def solve_case(a: list[int], d: int) -> int:
    lo = max(a)
    hi = sum(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if groups_needed(a, mid) <= d:
            hi = mid
        else:
            lo = mid + 1
    return lo

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        d = data[idx + 1]
        idx += 2

```

```
        a = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(a, d)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

3.10 Tests that matter

```
d = 1 -> answer is sum(a)
d = n -> answer is max(a)
large values -> 64-bit sums
two huge endpoint values with small middle values
```

4 C. Active Colors

4.1 Restatement

Given colored half-open intervals $[l, r)$ and query points, output for each query the number of distinct colors with at least one active interval at that point.

4.2 Constraints and traps

Checking every interval for every query is too slow.

Common traps:

- treating $[l, r)$ as closed;
- counting intervals instead of distinct colors;
- printing answers in event-sorted order;
- losing colors with multiple active intervals.

4.3 Observations

This is a sweep-line problem with a frequency map.

Events:

```
kind 0: interval end at r
kind 1: interval start at l
kind 2: query at x
```

Tie order:

```
end -> start -> query
```

This models half-open intervals because ending intervals are removed at x before the query, while starting intervals are added before the query.

Maintain:

```
freq[color] = number of active intervals of that color
distinct = number of colors with positive frequency
```

4.4 Sample walk-through

Intervals:

```
[1,5) color 10
[2,6) color 20
[4,7) color 10
[7,9) color 30
```

At $x = 4$, active intervals have colors 10,20,10, so the distinct color count is 2.

At $x = 7$, the $[4,7)$ interval has ended and $[7,9)$ has started, so the active color set is $\{30\}$.

4.5 Algorithm

```
events = []
for each interval:
    add (r, end, color)
    add (l, start, color)
for each query with id:
    add (x, query, id)

sort events by (coordinate, kind)

freq = empty map
distinct = 0
ans = array of q

for event:
    if end:
        decrease color frequency
        if it becomes zero, distinct -= 1
    if start:
        if color frequency was zero, distinct += 1
        increase color frequency
    if query:
        ans[id] = distinct
```

4.6 Correctness proof

Lemma 1. When a query at coordinate x is processed, `freq[color]` equals the number of active intervals of that color at x .

Proof. Any interval ending at x has event kind 0, so it is removed before the query. This matches $[l,r)$, which excludes r . Any interval starting at x has event kind 1, so it is added before the query. This matches $[l,r)$, which includes l . Intervals with smaller endpoints have already been processed, and intervals with larger endpoints have not. $\square$

Lemma 2. At every point in the sweep, `distinct` equals the number of colors with positive active frequency.

Proof. On a start event, `distinct` increases exactly when the previous frequency was zero. On an end event, it decreases exactly when the frequency becomes zero. Other changes do not affect whether the color is present. `\square`

Theorem. The algorithm returns the correct answer for every query in original order.

Proof. By Lemma 1, the frequency map is correct at each query coordinate. By Lemma 2, `distinct` is exactly the number of active colors. The query's original id stores the answer in the correct output slot. `\square`

4.7 Complexity

There are $2n + q$ events.

```
Time:  $O((n+q) \log(n+q))$   
Memory:  $O(n+q)$ 
```

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Event {
    long long x;
    int kind;
    long long color;
    int idx;
};

vector<int> solve_case(const vector<tuple<long long, long long, long long>> &intervals, const vector<int> &queries) {
    vector<Event> events;
    events.reserve(intervals.size() * 2 + queries.size());
    for (auto [l, r, color] : intervals) {
        events.push_back({r, 0, color, -1});
        events.push_back({l, 1, color, -1});
    }
    for (int i = 0; i < (int)queries.size(); ++i) {
        events.push_back({queries[i], 2, 0, i});
    }
    sort(events.begin(), events.end(), [](const Event &a, const Event &b) {
        if (a.x != b.x) return a.x < b.x;
        return a.kind < b.kind;
    });

    unordered_map<long long, int> freq;
    int distinct = 0;
```

```

vector<int> ans(queries.size());
for (const Event &event : events) {
    if (event.kind == 0) {
        int count = freq[event.color] - 1;
        if (count == 0) {
            --distinct;
            freq.erase(event.color);
        } else {
            freq[event.color] = count;
        }
    } else if (event.kind == 1) {
        int count = freq[event.color];
        if (count == 0) ++distinct;
        freq[event.color] = count + 1;
    } else {
        ans[event.idx] = distinct;
    }
}
return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        vector<tuple<long long, long long, long long>> intervals;
        intervals.reserve(n);
        for (int i = 0; i < n; ++i) {
            long long l, r, color;
            cin >> l >> r >> color;
            intervals.push_back({l, r, color});
        }
        vector<long long> queries(q);
        for (long long &x : queries) cin >> x;
        vector<int> ans = solve_case(intervals, queries);
        for (int i = 0; i < q; ++i) {
            if (i) cout << ' ';
            cout << ans[i];
        }
        cout << '\n';
    }

    return 0;
}

```

4.9 Full Python solution

```
import sys

def solve_case(intervals: list[tuple[int, int, int]], queries: list[int]) -> list[int]:
    events: list[tuple[int, int, int, int]] = []
    for l, r, color in intervals:
        events.append((r, 0, color, -1))
        events.append((l, 1, color, -1))
    for idx, x in enumerate(queries):
        events.append((x, 2, 0, idx))
    events.sort()

    freq: dict[int, int] = {}
    distinct = 0
    ans = [0] * len(queries)
    for _, kind, color, idx in events:
        if kind == 0:
            count = freq[color] - 1
            if count == 0:
                distinct -= 1
                del freq[color]
            else:
                freq[color] = count
        elif kind == 1:
            count = freq.get(color, 0)
            if count == 0:
                distinct += 1
            freq[color] = count + 1
        else:
            ans[idx] = distinct
    return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        q = data[idx + 1]
        idx += 2
        intervals: list[tuple[int, int, int]] = []
        for _ in range(n):
            l = data[idx]
            r = data[idx + 1]
```

```

        color = data[idx + 2]
        idx += 3
        intervals.append((l, r, color))
        queries = data[idx:idx + q]
        idx += q
        out.append(" ".join(map(str, solve_case(intervals, queries))))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4.10 Tests that matter

```

query exactly at l
query exactly at r
multiple active intervals with same color
duplicate query points
queries before all intervals and after all intervals

```

5 Stress testing

For Problem A, brute force every subarray.

For Problem B, brute force all split positions or use a small dynamic program.

For Problem C, answer every query by scanning all intervals and building a set of active colors.

The local generators use exactly these oracles.

6 What to carry forward

Core-pattern contests are won by disciplined recognition:

- contiguous and maintainable -> sliding window;
- monotone feasibility -> binary search;
- endpoints and active state -> sweep line;
- sorted queries -> offline processing.

The next phase starts graphs. The same habit remains: read constraints, model the state, choose the tool, then prove the invariant.